Chapter 05: Spring Boot

Department of Software Engineering
Faculty of Information Technology (HCM-UTE)
Course: Web Programming
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

Content

- Spring Boot overview
- Spring Boot request flow
- Tools to create Spring Boot project
- Default Project structure
- Concepts

Spring Boot

From raw Servlet/JSP to modern MVC with auto-configuration

Traditional Java web apps

- Hand-written web.xml, WAR, external Tomcat
- Controller = Servlet
- View = JSP
- Heavy XML and many deploy steps
- MVC is correct but the number of boilerplate code is large

What Spring Boot adds

- Starters: web dependencies bundled (spring-boot-starter-web)
- Auto-configuration: sensible defaults
- Embedded Tomcat: run from main()
- Still applies the MVC architecture.
- Official documentation: https://spring.io/projects/spring-boot

Versions

All projects

- Micrometer
- Project Reactor
- Spring Boot
- Spring Framework
- Spring Data
- Spring Cloud
- Spring AI
- Spring gRPC
- Spring Security
- Spring for GraphQL
- Spring Session
- Spring Integration
- Spring HATEOAS
- Spring Modulith
- Spring REST Docs
- Spring Batch

Spring Boot 4.1.1

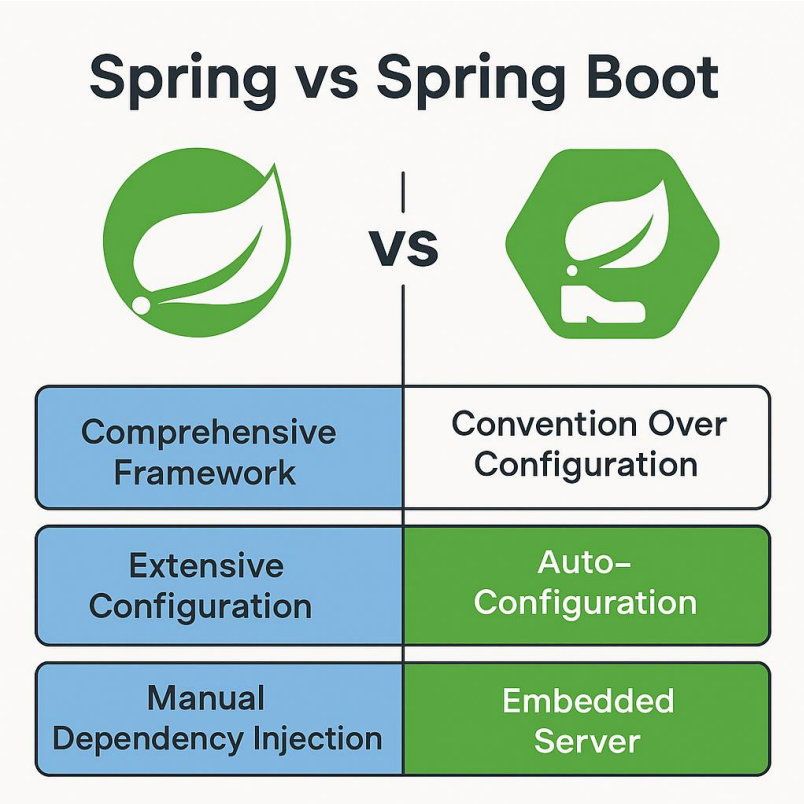
OVERVIEW | LEARN | SUPPORT | SAMPLES

Branch	Initial Release	End of OSS Support	End Enterprise Support *
4.2.x	2026-11	2027-12	2028-12
4.1.x	2026-06	2027-07	2028-07
4.0.x	2025-11	2026-12	2027-12
3.5.x	2025-05	2026-06	2032-06
3.4.x	2024-11	2025-12	2026-12
3.3.x	2024-05	2025-06	2026-06
2.7.x	2022-05	2023-06	2029-06

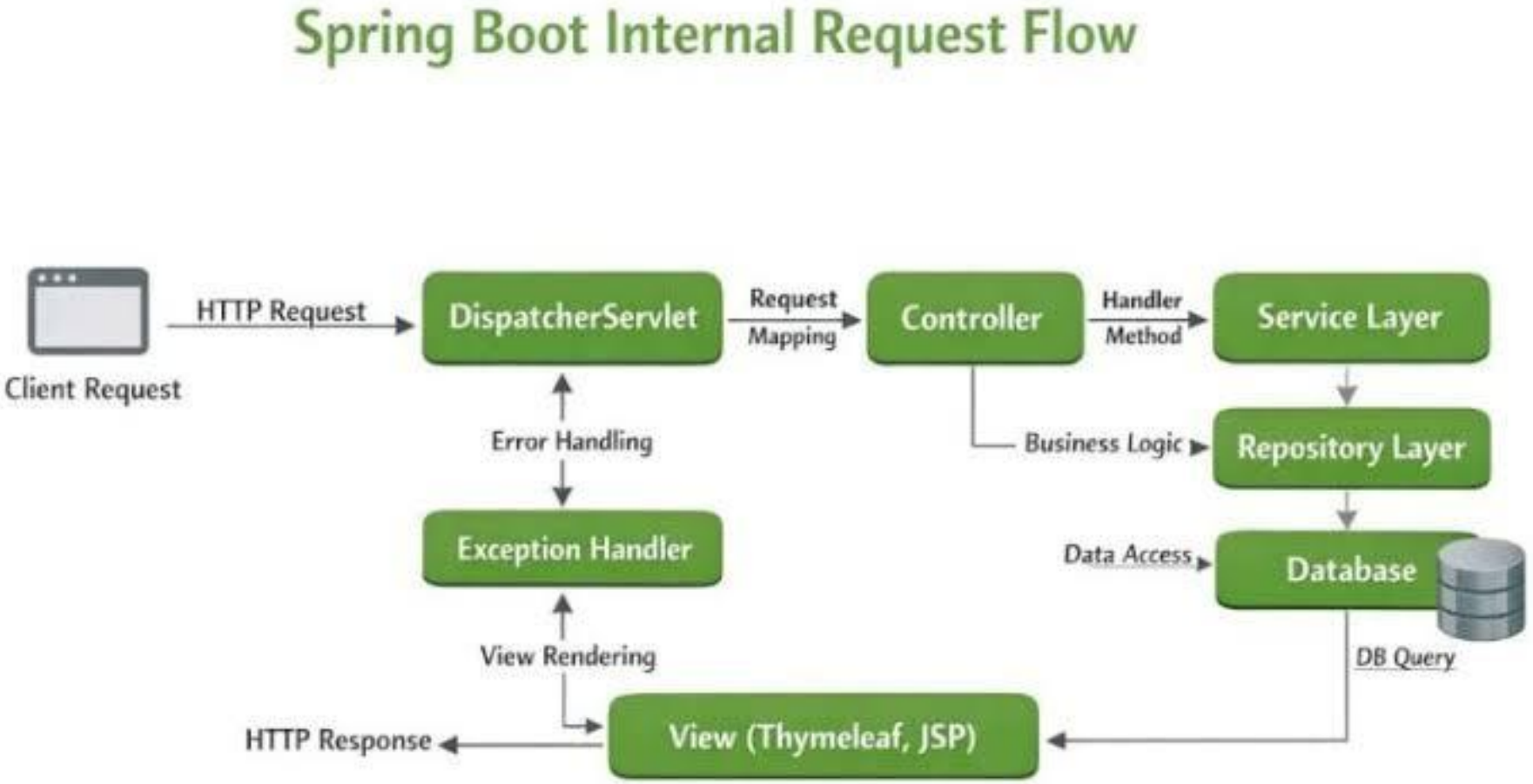
More

Spring Framework vs Spring Boot

- Spring Framework: IoC, AOP, Spring MVC, transactions…
- Spring Boot: Spring Framework + convention + starters
- Spring Boot does not replace MVC, it wires it



Spring Boot request flow



Create Spring Boot project

- Use Visual Studio Code extension
  - Spring Boot Extension Pack
  - Refer to chapter 04 at Demo section for more detail

- Use the official tool Spring Initializr: https://start.spring.io/

Common starters

- spring-boot-starter-web: MVC + Tomcat + Jackson
- spring-boot-starter-thymeleaf: HTML views
- spring-boot-starter-data-jpa: JPA/Hibernate
- spring-boot-starter-test: JUnit on test classpath



Project structure

Convention over configuration

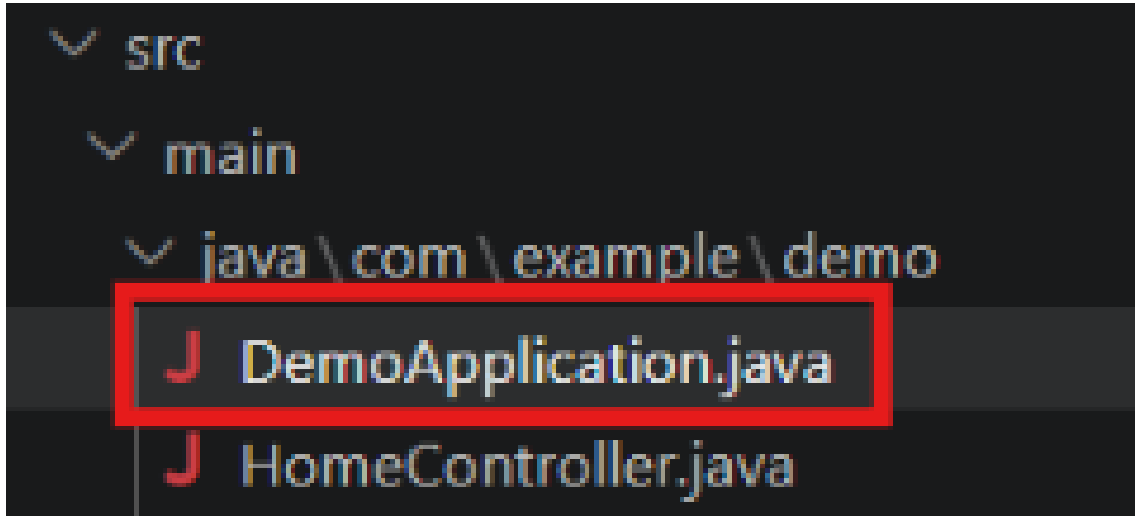
Project structure

```
project/
├── src/
│   ├── main/                          # production code
│   │   ├── java/                      # Application class
│   │   └── resources/                 # classpath resources
│   │       ├── application.properties  # app config
│   │       ├── templates/             # Thymeleaf views
│   │       └── static/                # CSS, JS, images
│   └── test/                          # test root
│       └── java/                      # unit tests
└── pom.xml                            # starters + versions
```

@SpringBootApplication

@SpringBootApplication

- The class with main() boots the whole app
- @Configuration: declare beans
- @EnableAutoConfiguration: turn on defaults
- @ComponentScan: scan beans from the root package




```java
package com.example.demo;

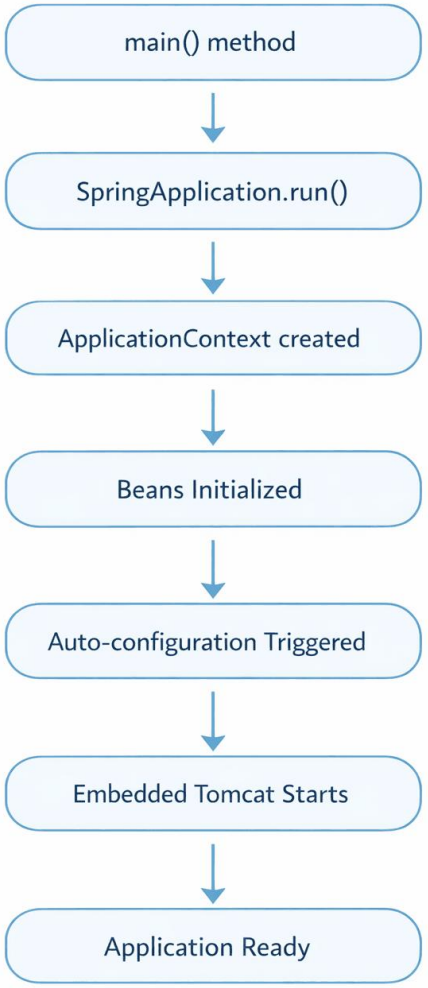
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

Startup flow



IoC and beans

Controllers do not new a Service

Inversion of Control

- The container creates and wires dependencies
- Dependency Injection: receive, do not use the `new` keyword.
- Layers: Controller – Service – Repository
- Swap the Model when JDBC/JPA changes

Stereotype annotations

- @Controller / @RestController: web layer
- @Service: business logic
- @Repository: data access
- @Component: generic bean

Dependency Injection types

- Field injection
- Setter injection
- Constructor injection

Field injection (not recommended)

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    // Field Injection: Spring uses reflection to inject the dependency directly into the field.
    // Pros: Concise syntax.
    // Cons: Cannot be declared as final, hides dependencies, and makes unit testing harder.
    @Autowired
    private UserService userService;

    public String getUser() {
        return userService.getUserName();
    }
}
```

Setter injection (rarely used)

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    private UserService userService;

    // Setter Injection: Spring calls this setter method to inject the dependency after object creat
ion.
    // Pros: Allows dependencies to be changed or re-injected at runtime.
    // Cons: Object can be created without its dependencies (incomplete state), risking NullPointerE
xception.
    @Autowired
    public void setUserService(UserService userService) {
        this.userService = userService;
    }

    public String getUser() {
        return userService.getUserName();
    }
}
```

Constructor injection (recommended)

```java
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    // Constructor Injection: Dependencies are declared as final fields.
    // Pros: Ensures the object is always in a fully initialized state, supports immutability,
    // and makes unit testing straightforward (pass mocks via constructor).
    // Note: @Autowired is optional here if there is only one constructor (Spring 4.3+).
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    public String getUser() {
        return userService.getUserName();
    }
}
```

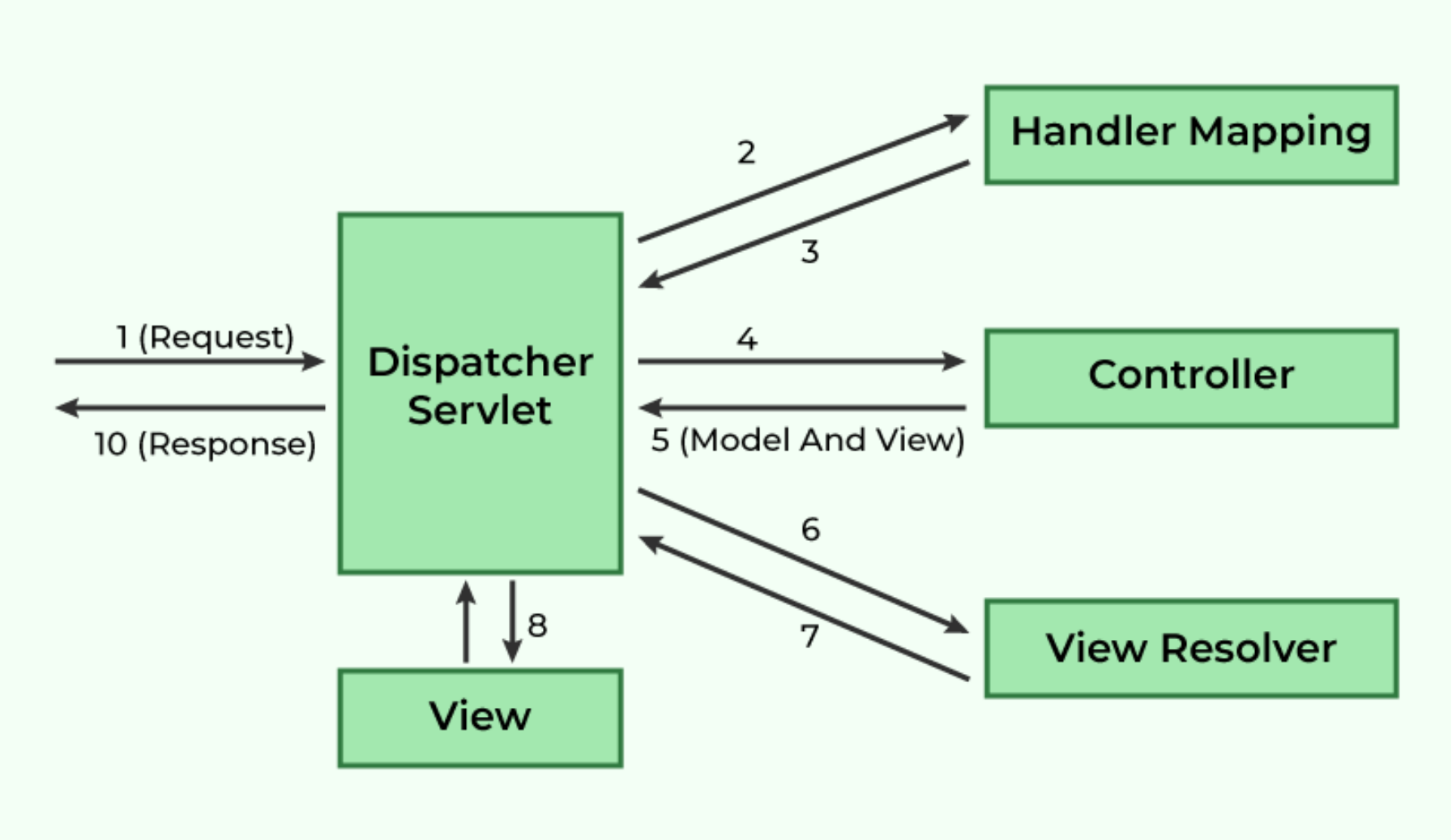
Spring MVC

DispatcherServlet is the front controller

DispatcherServlet

- One central Servlet receives every HTTP request
- Maps URL → controller method
- Binds the Model, then picks View or JSON
- Still a Servlet, same J2EE idea as before

Spring MVC request flow



@Controller vs @RestController

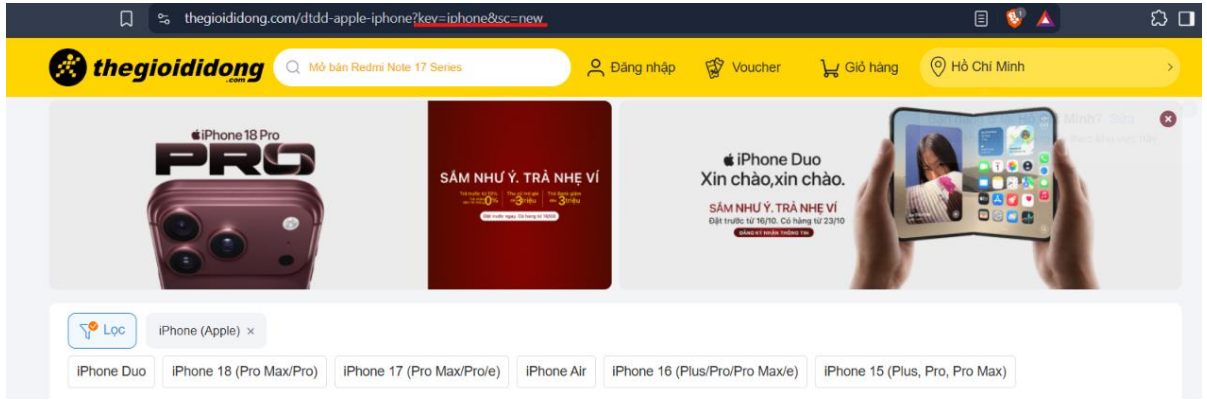
- @Controller: return a view name (HTML)
- @RestController = @Controller + @ResponseBody
- @RestController: return JSON/XML directly
- Choose by page vs Web API

HTTP mappings

- @RequestMapping: base path and method
- @GetMapping: read a resource
- @PostMapping: create / submit a form
- @PutMapping / @DeleteMapping: update / delete

Request parameters #1

- @RequestParam: query / form (HttpRequest)
  - Example: https://example.com/search?query=iphone&sort=asc



- @PathVariable: value in the URL path
  - Example:
    - https://example.com/users/123
    - https://example.com/api/v1/users/123

Request parameters #2

- @RequestBody: JSON → Java object
  - Example: Client send POST request with **body**:

```
{
    "name": "Nguyen Van A",
    "email": "a@example.com"
}
```

- @RequestHeader: read an HTTP header
  - Example: Client send request with **header**

```
Authorization: Bearer abc123token
User-Agent: Mozilla/5.0
```

View, config, data

Model binds the view; properties replace web.xml

Model and View

- `Model.addAttribute`: pass data to the view



```java
@Controller
public class HomeController {
    @GetMapping("/")
    public String index(Model model) {
        model.addAttribute("message", "Hello world");

        return "index";
    }
}
```

- Thymeleaf: templates/*.html (default convention)
- ViewResolver picks the template by view name

src/main/resources/application.properties file

- server.port: embedded Tomcat port
- spring.datasource.*: JDBC URL, user, password
- No web.xml needed for basic setup
- Profiles: application-dev.properties, …
  - application-dev.properties: used during development (dev).
  - application-staging.properties: used when double-check before go to the Production environment.
  - application-prod.properties: used when running in production.
  - application-test.properties: used when running tests.
  - Activate: spring.profiles.active=dev : Spring Boot automatically loads the appropriate configuration file. This eliminates the need to modify your code when switching between environments.

Summary: Servlet/JSP MVC → Spring Boot

- HttpServlet → @Controller / @RestController
- JSP → Thymeleaf (or keep JSP)
- web.xml → annotations + application.properties
- Installed Tomcat → embedded Tomcat
- Spring Boot cuts the wiring. The principle stays MVC on Servlet
  - Controller coordinates
  - Model holds data/DB
  - View only renders.

Q&A – Thank you